BabyScholar HW2: guided test generation and evaluation

November 8, 2026 · Branch: `hw2/initial-baseline`
Prepared by the coding agent. The student must review the reflection and submission details before submitting.

1. Test Generation Approach

BabyScholar is a React and Cloudflare Worker research workspace with arXiv search, a persistent paper library, PDF upload/extraction, metadata, summaries, and questions. The objective was to generate tests, evaluate them, and use actual results to guide improvements. Coverage is evidence of execution, not proof that assertions are correct.

The inherited initial suite contained 24 deterministic Node/Vitest tests in two files: 13 identity/link cases and 11 question-request cases. It targeted modern and legacy arXiv IDs, version handling, unsafe links, cross-paper mismatch, request shape, normalized question identity, and UTF-8 limits. The initial interaction record preserves the instruction: “We also want profiling and code coverage analysis. Everything to know where we are at right now.” That generation and assessment are distinct from the later guided improvement rounds.

The user then requested two separate improvements based on measured gaps, preservation of prompts/results, and diagnosis of stale tests and environment failures. The request and six-section rubric are retained verbatim. The later extension request asked for stronger workflow coverage, bounded browser diagnosis, selected fault probes, and a verified PDF. The user also identified arXiv's injectable fetch, clock, sleep, timeout and response-size seams.

Tests use independent expected values, real Web Request/Response/stream objects where appropriate, injected clocks, and deterministic fixtures. React tests use Testing Library roles, labels and user interactions; only HTTP boundaries are mocked. Unhandled routes fail even if the UI catches their rejection. Timeout tests use abort-aware requests; stale-response tests deliberately resolve old requests after cancellation. No live arXiv or OpenAI calls are needed.

The final suite contains **260 tests in 10 test files**: 200 Node tests and 60 React component tests in jsdom. Existing browser/Worker integration scripts are separate evidence, not additional Vitest tests. Source was unchanged through rounds 1 and 2. Round 3 confirmed one existing App bug and made a separate, narrow product fix; other changes concern tests, tooling and reports.

2. Initial Test Evaluation

The Windows reproduction started from initial-test commit `e2f5c97` on the HW2 branch. `npm.cmd ci` completed, all 24 tests passed, and 17/19 baseline stages passed. The aggregate baseline exited 1. Initial coverage was:

Metric	Covered / total	Coverage
Statements	91 / 2960	3.07%
Branches	64 / 2595	2.46%
Functions	13 / 533	2.43%
Lines	86 / 2444	3.51%

The metadata verifier expected prompt v2 instead of current v4 and mandatory source-matching quotes. Its bibliographic fields, hash and page selection actually matched. The Windows browser launched but waited for obsolete summary progress wording. The historical Linux browser-discovery failure was a different environment limitation. These were not counted as product defects. **No existing product bug was confirmed in the initial evaluation or first two improvement rounds.**

The initial tests correctly asserted their narrow contracts. Identity tests used explicit expected IDs, not merely agreement between implementations. However, some rejection cases asserted only status, and important streaming, recovery, source-selection, storage/routing and UI behavior was unmeasured. High coverage of one small parser did not imply broad application coverage. The initial assessment links raw logs, per-file coverage and exact snapshots. Original Linux evidence remains separate from the Windows comparison.

3. Iterative Test Improvement

Round 1 — request boundaries and output contracts. The recorded instruction targeted actual gaps: "actual bytes over dishonest Content-Length, stalled/erroring streams, the shared 10-second read deadline, cancellation that never settles, and released stream locks." The agent added 34 request/stream/upload tests and 48 output-validation tests, bringing the total to 106.

Assertions cover both sides of byte boundaries, an independent fixed PDF hash, Unicode code points versus UTF-16 units, nullable metadata, and ordinary prose. The app does not require citations for new summaries or answers. New unverified quotes are discarded; older saved quotes remain readable. Stale verifiers were aligned with those contracts. All 106 tests passed and line coverage rose to 12.31%. The baseline passed 18/19 stages, but the browser timed out at a later persisted-library refresh.

Five selected mutations were applied only in disposable copies. Two were detected; lossy UTF-8 decoding, retrying a nonretryable generation failure, and exceeding a summary source budget by one byte survived. The UTF-8 tests could pass for the wrong reason because their fixtures were also invalid JSON.

Round 2 — recovery, source budgets and surviving-fault feedback. The second instruction used those results and remaining uncovered modules. It directed tests of the shared recovery state machine and source selection, and invalid UTF-8 bytes "inside otherwise valid JSON strings." The agent added 27 recovery, 20 source-selection and 10 UTF-8 integrity cases.

These assert shared deadlines, remaining correction budgets, a two-attempt limit, ownership checks, no retry of transport/provider failures, sanitized errors, distinct metadata/summary byte budgets, intact Unicode prefixes and unchanged persisted extraction. Valid U+FFFD is an encoding positive control. An extra parenthesis initially prevented the 10 new encoding cases from loading; the failed file and run are preserved. Correcting the test syntax yielded 163/163 passing tests and 16.12% line coverage. All five selected faults were detected. Browser readiness still failed, with a successful direct Worker GET and a timed-out Vite GET.

Round 3 — observable workflows and arXiv resilience. The extension first verified branch/remote commit `d66051c`, ran a fresh install and baseline, and preserved another 163-test, 18/19-stage result. App and DocumentWorkspace accounted for 807/2444 measured lines (33.02%) at zero Vitest coverage; arXiv contributed another 214 uncovered lines. This did not mean they were never exercised by the separate legacy browser checks.

The pre-implementation self-instruction said: "Include saved-paper identity, uncertain-save GET reconciliation without automatic POST retries, loading/empty/error/malformed states, duplicate clicks, stale library/detail responses, explicit PDF summary/question actions, retained results after failures, and cleanup." It also required a frozen-suite bridge before new cases. The user's arXiv guidance was verified in `ArxivSearchOptions` and used directly.

Pinned DOM testing dependencies and TSX discovery were added. Before new tests, the unchanged 163 cases reproduced **exactly the round-2 coverage counts** under the new configuration. Then 37 arXiv cases and initially 56 component cases raised the suite to 256 passing tests and 55.64% line coverage. arXiv tests cover query normalization, Atom parsing, failures, coalescing, independent returned data, TTL expiry, eviction, queue recovery, byte bounds and timeouts. Component tests cover search/save/reopen/remove, stale responses, PDF processing, explicit generation, retained results, uploads, downloads and deletion.

Actual failures changed the tests. A bare legacy ID became a valid Atom-entry URL. A button query used the wrong accessible name. A summary test incorrectly assumed automatic reconciliation of an in-progress response; the contract waits for an explicit GET reload. Typechecking caught unsupported query options. These were test/tooling errors, distinct from the save bug. The development record preserves failures, corrections and snapshots without inventing additional user interactions.

Eight workflow mutations were recorded before execution. The 256-test control passed; seven faults were detected. The survivor rejected an exactly 10 MiB downloaded PDF: upload-boundary tests did not cover the download reader. Feedback added a download boundary pair, stale-download protection, and a negative counterpart to save reconciliation, producing 260 tests. The same eight faults were then detected. Line coverage stayed 55.64%, while branch coverage and assertion strength improved.

Browser investigation. Draining child output did not resolve initial-page stalls. Diagnostics separated API/static requests and fresh HTTP connections; Vite API and static requests could stall while Worker requests succeeded. A standalone Vite/module/proxy probe passed all 21 requests. No application proxy change or longer readiness wait was introduced.

Further runs exposed three stale harness assumptions: old Q&A progress text; expecting metadata to fail without configuring a provider failure; and routing ordinary-text summary/Q&A requests by removed schema names. Repairs used current UI text, an explicit loopback failure fixture, and existing loopback sentinel keys. The complete built-app preview run passed the same assertions, including real local persistence, processing, generation, download and deletion. That validates the preview mode; it does not establish the cause or resolution of intermittent development-server stalls.

4. Final Evaluation

The final suite has **260 passing tests**. The full baseline outcome is **19/19 stages passed (aggregate exit 0)**. A separately recorded built-app browser preview run passed; it remains distinct from the normal development-server baseline stage.

Measure	Initial	Round 1	Round 2	Final round 3
Passing Vitest tests	24/24	106/106	163/163	260/260
Statements	3.07%	12.29%	15.87%	53.09%
Branches	2.46%	10.13%	12.21%	51.44%
Functions	2.43%	12.57%	15.38%	57.11%
Lines	3.51%	12.31%	16.12%	55.64%
Confirmed existing product bugs	0	0	0	1, fixed

Final counts are 1572/2961 statements, 1335/2595 branches, 305/534 functions and 1360/2444 lines. Line coverage increased by 52.13 percentage points from the initial suite. Target-file line coverage is 223/227 (98.23%) for App, 531/580 (91.55%) for DocumentWorkspace, and 207/214 (96.72%) for arXiv. The all-file CSV and machine-readable comparison retain every phase, the bridge, exact counts, commands and remaining uncovered files.

This is one combined instrumented Vitest 5.1.0/V8 measurement across Node and jsdom tests, not averaged frontend/backend percentages. Scope remains all `src/**/*.ts(x)` except declarations, type-only `*-types.ts`, and two local probe modules. Scripts, CSS and fixtures are excluded. The frozen bridge matches round 2; existing dependency versions were retained. The App fix adds one statement and one arrow function, changing those denominators from 2960/533 to 2961/534. Lines and branches retain the same denominators. Browser/workerd subprocess execution is excluded from V8 coverage.

One genuine existing bug, fixed separately. If an untrustworthy save response is followed by a GET confirming the paper, App displayed both “Saved” and “The save could not be confirmed.” Four variants reproduced it against unchanged source. Reconciliation normalized an already-prefixed identity twice and rejected the confirmed record. The one-line fix compares normalized identities directly. Regression tests require one POST only and reject a different paper as confirmation. This is a feedback/state bug; no data loss or duplicate storage is claimed. The bug record contains reproduction, oracle, cause and failing evidence. Product fix commit: `852eb2c`; test/tooling commit: `2c83268`.

Selected faults, not a general mutation score. The original five seeds progressed from 2/5 detections in round 1 to 5/5 in round 2 and 5/5 in the final rerun. The eight new seeds progressed from 7/8 to 8/8 after refinement. They target TTL expiry, shared author arrays, a poisoned request queue, stale library reads, stale document details, lost summaries after reload failure, leaked object URLs and an off-by-one download limit. Each experiment has a passing clean control and records failing test names. These deliberately selected artificial faults are not discovered product bugs and do not estimate effectiveness against arbitrary mutations or real-world defects.

Limits remain substantial. About 44% of source lines remain unmeasured by this suite, including much Worker routing, storage, provider-adapter and PDF parser code. Existing offline integration checks add evidence, not combined coverage. jsdom does not prove layout, browser download permission, hosted authentication or network behavior. The preview run forced download permission in its disposable browser profile; default user download behavior is unverified. Development-server stalls remain unexplained. No live model quality, production load, OCR or deployment was tested. CPU profiles and stage timings are retained; differing workloads and local load do not support a performance claim.

5. Reflection

Agent-authored reflection for the student's review. Revise it to reflect personal experience; it does not claim independent student testing or audit.

The agent was effective at turning explicit contracts into boundary, recovery and observable workflow tests. Context mattered: the citation/version warning prevented obsolete requirements from becoming oracles, and the arXiv seam guidance enabled realistic time and failure scenarios without live traffic. Source contracts and execution feedback were more reliable than old labels.

The agent also made mistakes. Initial UTF-8 cases could pass for the wrong reason. Later it used an incorrect Atom ID, a wrong button name and a mistaken summary reconciliation assumption. Syntax and query-option errors required correction. An agent-generated test is therefore a hypothesis about the contract, not an authority. Failed snapshots and expected/actual comparisons made corrections reviewable instead of hiding them behind a final green run.

The clearest improvements came from specific feedback: put invalid bytes inside valid JSON; preserve a confirmed save after a stale read; test download limits separately from upload; and assert absence of contradictory feedback as well as presence of success. The save bug required that stronger oracle. Browser failures instead required distinguishing stale fixtures from local infrastructure; repeated waits alone would not explain them.

Human input consisted of task scope, contract warnings, the six-section rubric and the arXiv suggestion. The agent selected and implemented tests, corrected its mistakes, repaired fixtures and made the narrow documented bug fix. No user-authored test edits or independent human review are claimed. The student should review this analysis, personalize the reflection, confirm course identifiers/deadline, and submit the reviewed PDF through the course channel.

6. GitHub Repository

Repository: panappin/babyScholar.
Branch: hw2/initial-baseline.
Final suite: tests/baseline (10 test files, two fixture/helper files). Instructions, results, diagnostics, coverage, snapshots and SHA-256 manifests are in docs/hw2/rounds. The round-3 assessment indexes new evidence.

Tested: Node 24.11.1, npm 11.6.2, Windows x64, Intel i9-13900HX. Use the locked dependencies; jsdom is pinned to 27.4.0 for this Node version. From the HW2 branch:

```
npm.cmd ci
npm.cmd test
npm.cmd run test:coverage
npm.cmd run test:workflows:types
npm.cmd run baseline
```

`test:workflows:types` checks the three new workflow files and their helpers. The baseline separately typechecks production browser and Worker projects. On other platforms use `npm` instead of `npm.cmd`; the legacy browser script requires Chrome/Edge in its supported Windows paths. Coverage writes HTML, JSON and LCOV to `coverage/baseline/`. The baseline creates a timestamped `artifacts/baseline/` directory, retains each stage log and exit status, and exits nonzero if any stage fails. Do not reuse an evidence output directory.

Additional reproduction commands:

```
node scripts/baseline/compare-hw2.mjs
node scripts/baseline/compare-workflows.mjs
node scripts/baseline/workflow-fault-probes.mjs refined
node scripts/baseline/fault-probes.mjs round-3
npm.cmd run build:frontend
node scripts/baseline/browser-experiment.mjs preview
node scripts/baseline/render-report.mjs
```

Faults run in disposable source/test copies. Preview uses the built app and the same browser assertions with disposable local Worker/D1/R2 state and loopback provider fixtures. It is separate from normal baseline mode. These checks require no credentials or live provider calls. The PDF renderer uses locked Marked and local Chrome/Edge, converts relative links to branch URLs, and writes `docs/hw2/report.pdf` from this editable source. Text, links and rendered pages are verified separately before delivery.

The code, evidence and report are pushed to the HW2 branch. No merge to `main` or assignment submission is performed. The student must review and submit the PDF and confirm the repository deadline requirement.